Modular

# Modular Tech Talk Series: Kernels

Ben/Jiexiang/Yingbo

# Overview


01 Kernel library comparison

02 Hardware overview

03 Hardware bring up

04 Tooling

05 Fusion

06 Where we are heading

07 Agentic Skills

# What are kernels @ Modular

### What is a kernel?

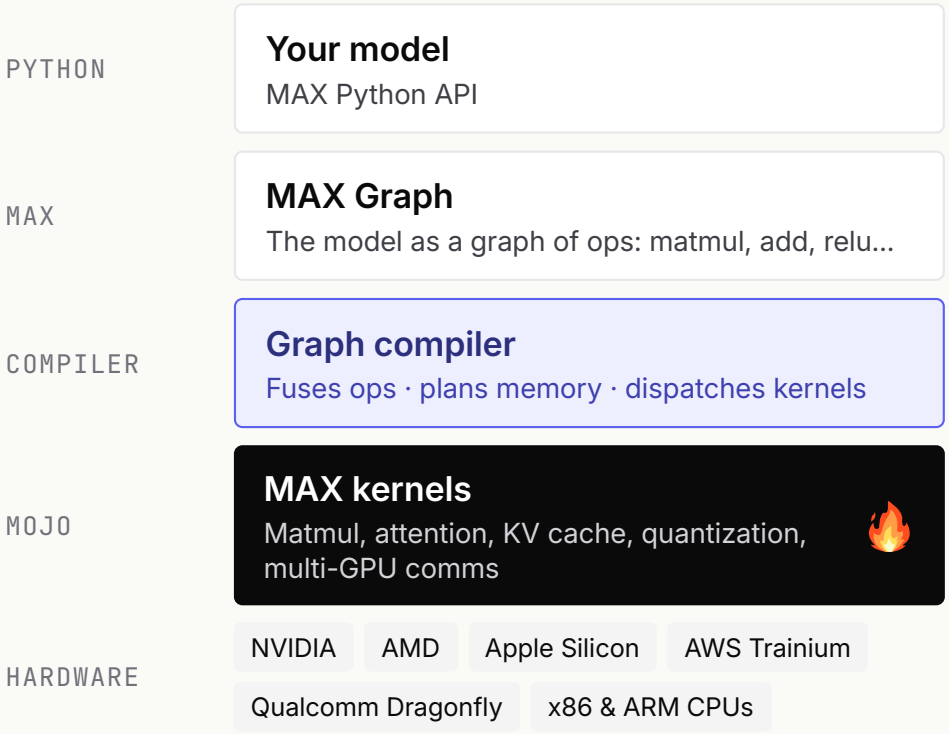
A program that runs in parallel on a GPU, CPU, NPU or other accelerator: a matrix multiply, attention or several ops fused together

### Graph compiler

Optimizes the model as a whole: fuses ops, plans memory, and selects the right kernels for the hardware

### Mojo

The language every MAX kernel is written in, Python-style syntax, compiled to native code for each target



# Differences between MAX Kernels approach and others

| APPROACH | LANGUAGE | YOU PROGRAM | RUNS ON | TRADE-OFF |
|---|---|---|---|---|
| **CUDA C++** | C++ | Individual threads | NVIDIA | Full control, single vendor |
| **OpenCL** | C dialect | Individual threads | Many vendors | Portable, but tooling and peak performance lag |
| **Triton** | Python DSL | Tiles of data | NVIDIA, AMD, others | Productive; compiler hides low-level control |
| **cuTile** | Python | Tiles of data | NVIDIA | Tile model, single vendor |
| 🔥 **MAX / Mojo** | Python-style systems language | Tiles or threads, in one language | NVIDIA, AMD, Apple, AWS Trainium, Qualcomm Dragonfly, CPUs | Low-level control and portability; fused by the graph compiler |

04

# Benefits of the MAX kernel approach

01

### SOTA performance

Portable abstractions with hardware-specific paths where it matters.

02

### Portable abstractions

Allow for composable and reusable kernel code.

03

### Compile-time specialization

Kernel sources specialized for data-types and specific targets.

04

### Op fusion

Automatically fuse nearby ops where possible, or use custom fused ops.

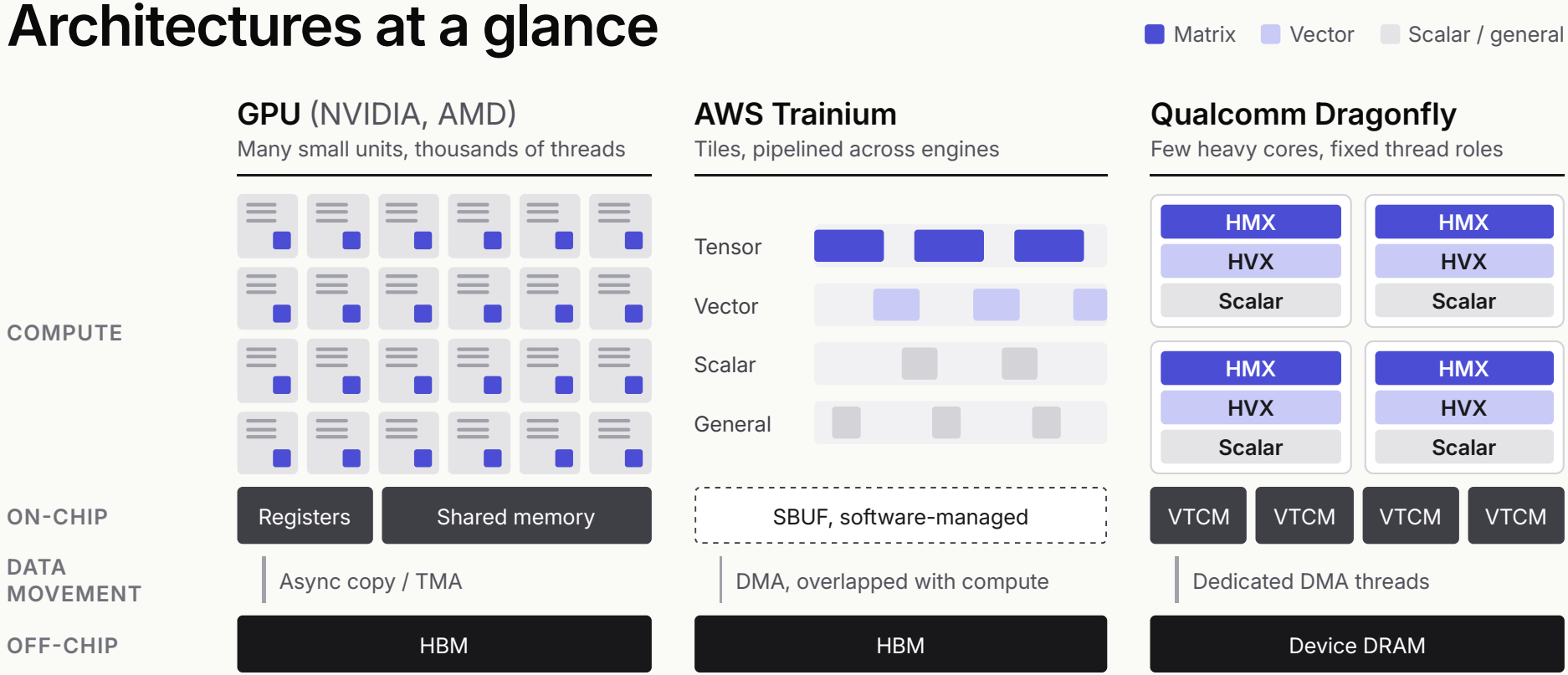
05

### Open and extensible

Porting to new hardware doesn't require new compiler passes.

05

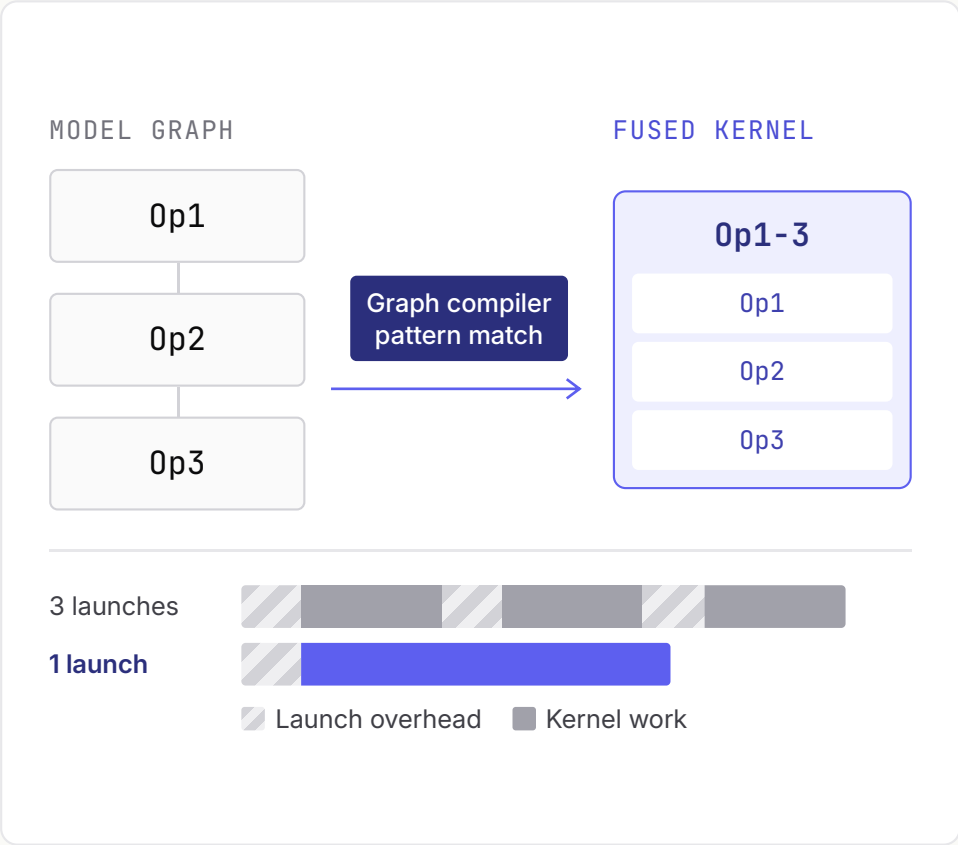
# Architectures at a glance



# Bringing up Dragonfly with the MAX approach

## Op fusion

- The graph compiler recognizes common op patterns and replaces them with a single fused kernel
- The pattern is target-independent: Only need to register a kernel for the fused op
- Three ops become one launch, which matters on hardware where launches are expensive



# Bringing up Dragonfly with the MAX approach

## Compile-time specialization: one source for every Dragonfly generation

- Each chip's cores, threads and VTCM size are known to kernels at compile time
- Kernels branch on them at compile time, so only the matching code path is built
- The algorithm can change, not just constants: flash attention runs as two thread teams on gen-2 and one on gen-1

Mojo

```
# Per-chip facts, resolved at compile time
def num_hvx_threads() → Int:
    comptime if is_aic100(): return 4
    elif is_aic200():        return 8

# Two thread teams on gen-2, one on gen-1
comptime FA_NUM_HVX_TEAMS = (
    2 if num_hvx_threads() ⩾ 8 else 1
)
```

# Bringing up Dragonfly with the MAX approach

## Lowl-level control where needed

- Direct access to HVX intrinsics where the compiler alone doesn't produce the best code
- Each intrinsic is wrapped once as a typed Mojo function with compile-time checks, then reused across kernels
- Portable APIs choose the hardware path: exp in any kernel uses the Hexagon SDK's HVX routine on Dragonfly

Mojo

```
# Wrap once, with compile-time checks...
def fmac_sf_hf[dtype: DType, width: Int](
    c: SIMD[.float32, width],
    a: SIMD[dtype, width],
    b: SIMD[dtype, width],
) → SIMD[.float32, width]:
    comptime assert (dtype == .float16
                     and width == 64)
    return llvm_intrinsic[
        "llvm.hexagon.V6.vmpy.sf.hf.acc.128B",
        ...](c, a, b)
# ...then use it like any other function
acc = fmac_sf_hf(acc, av, bv)
```

# Inline Assembly

Building assembly string

type_definition.mojo

```mojo
@fieldwise_init("implicit")
struct UMMAKind(Equatable, Hashable, TrivialRegisterPassable, Writable):
    """Struct for UMMA instruction types.
    This struct defines the different types of UMMA instructions that is
    supported by BlackWell.
    """
    var _value: Int32
    comptime KIND_TF32 = Self(0)
    """TF32 type."""
    comptime KIND_F16 = Self(2)
    """F16 type."""
    comptime KIND_F8F6F4 = Self(3)
    """F8F6F4 type."""
    comptime KIND_I8 = Self(4)
    """I8 type."""
    comptime KIND_MXF8F6F4 = Self(5)
    """MXF8F6F4 type."""
    comptime KIND_MXF4 = Self(6)
    """MXF4 type."""
    comptime KIND_MXF4NVF4 = Self(7)
    """MXF4NVF4 type."""
```

inlined_assembly.mojo

```mojo
inlined_assembly[
    """{
        .reg .pred p;
        setp.ne.b32 p, $4, 0;
        tcgen05.mma.cta_group::2."""
    + String(kind)
    + scale_vector_size
    + """ [$0], $1, $2, $3, [$5], [$6], p;
    }""",
    NoneType,
    constraints="r,l,l,r,r,r,r",
](
    c_tmem,
    a_desc,
    b_desc,
    inst_desc,
    c_scale,
    sfa_tmem,
    sfb_tmem,
)
```

# Inline MLIR

Lowering Mojo constructs to hardware dialects

01 **MLIR Dialect Definition**

Trainium uses a MLIR dialect to define and execute low-level basic operations.

02 **No Direct Lowering**

Mojo language itself does not natively lower directly to the dialect.

03 **String Inlining**

Inlines custom strings formatted with types, memory access patterns, and execution flags.

inline_op.mojo

```
__mlir_op[
    `trn.dma_copy(`,
    `dst<%{tile_shape}>%{type_of(dst_mem)} %{dst_mem}[%{dst_ap}], `,
    `src<%{tile_shape}>%{type_of(src_mem)} %{src_mem}[%{src_ap}], `,
    `dge_mode=unassigned, oob_is_err=true) engine=dma`,
]
```

lowered_nisa.mlir

```
trn.dma_copy(
    dst<128| 512> = memref<128x512xbf16, #nisa.mem<sbuf>> %dst[d0, d1],
    src<128| 512> = memref<128x512xbf16, #nisa.mem<shared_hbm>> %src[d0,
    d1],
    dge_mode = unassigned,
    oob_is_err = true
) engine = dma
```

# Abstraction

TileTensor and Layout Primitives

## 01 TileTensor

The primary tensor abstraction used across hardware kernels for managing memory structures and tile-based operations.

## 02 Layout

- Layout algebra for defining spatial transformations
- Tiling and vectorization
- Swizzling optimizations tailored for GPU architectures

Block tiling GEMM · load stage 1/2

```
for k_i in range(ceildiv(K, BK)):
    var a_tile_dram = mat_a.tile[BM, BK](block_idx.y, k_i)

    copy_dram_to_sram_async[
        thread_layout=Layout.row_major(NUM_THREADS // BK, BK)
    ](a_tile_sram, a_tile_dram)

    var b_tile_dram = mat_b.tile[BK, BN](k_i, block_idx.x)

    copy_dram_to_sram_async[
        thread_layout=Layout.row_major(NUM_THREADS // BN, BN)
    ](b_tile_sram, b_tile_dram)

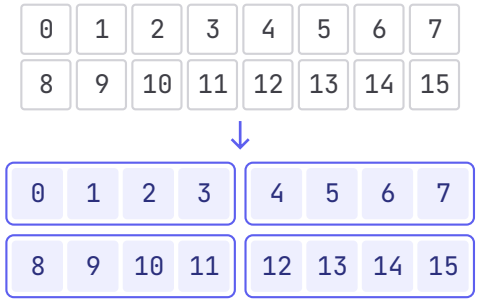
    async_copy_wait_all()
    barrier()
```

# TileTensor can express complex Layouts

## 02 TileTensor layout transforms allows for layout = shape : stride

### vectorize

`vectorize[1, 4]`

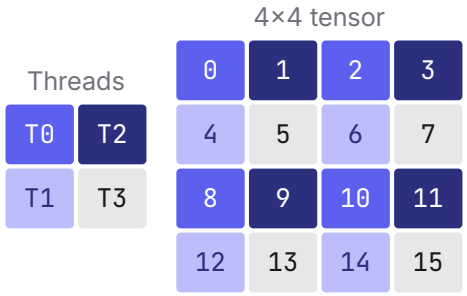



`(2,8):(8,1) → (2,2):(8,4) ∘ (1,4):(0,1)`

2×8 scalars become a 2×2 tensor of 1×4 SIMD vectors

### distribute

`distribute[(2,2):(1,2)]`




`T0 → (2,2):(8,2) = {0, 2, 8, 10}`

The thread layout tiles the tensor; each thread gets its own sub-layout

### swizzle

`Swizzle[2, 0, 2]`



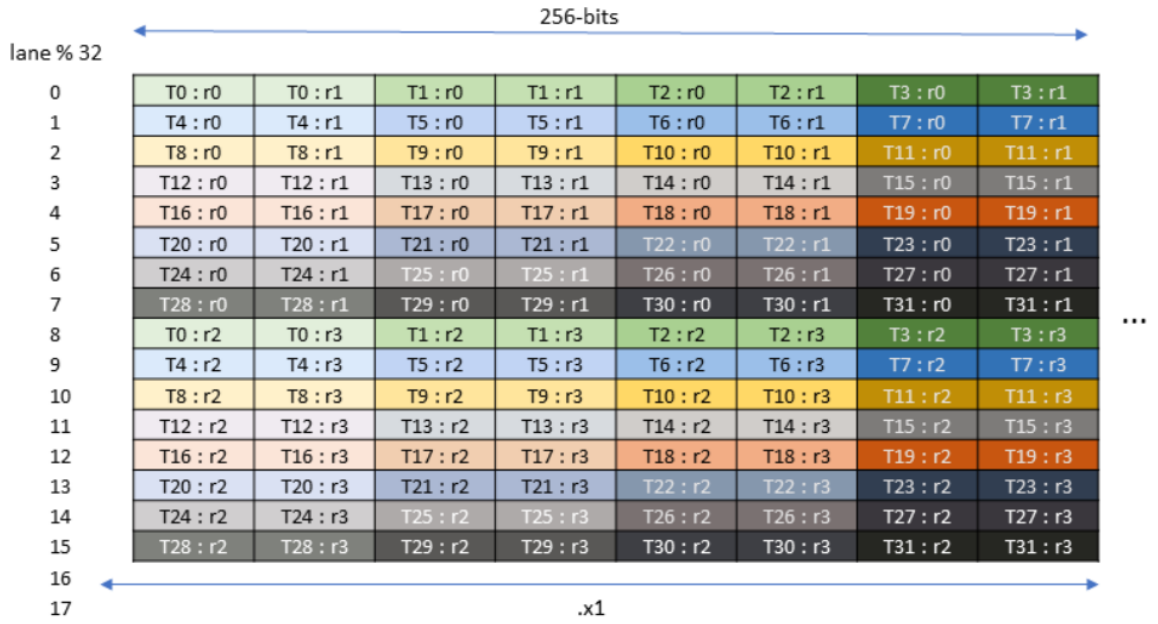

`i ^ ((i >> 2) & 0b11)`

Row bits XOR into column bits: chunk 0 of each row lands in a different bank

# TileTensor can express complex Layouts

01 **Hardware layouts**

- Rowmajor (CPUs)
- Tensorcore (NVIDIA)
- WMMA (AMD)
- Crouton (Dragonfly)




**16×256b: 4 registers spread over datapaths and cols**

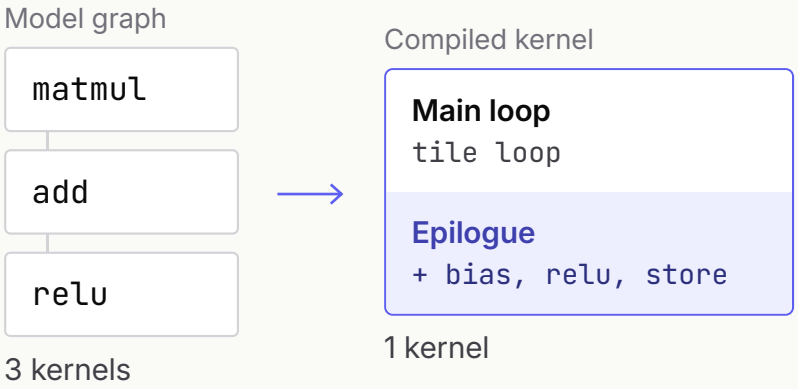
```
ɭ = ⟨v↦(2:ĉ)(2:8p̂); ℓ↦(4:2ĉ)(8:p̂)⟩
```

# Graph Compiler + Fusion

- Elementwise Fusion
- Registering an Op

# Elementwise Fusion

### Elementwise Fusion:




Without fusion, each combination is a separate hand-written kernel:

`Matmul_add` `Matmul_relu` `Matmul_add_relu` `matmul_gelu` `…`

The matmul kernel exposes an optional epilogue parameter

```
fn matmul[
    elementwise_lambda_fn: OptionalReg[
        elementwise_epilogue_type
    ] = None,
](c: TileTensor, a: TileTensor, b: TileTensor):
    # ... tiled main loop computes acc ...

    @parameter
    if elementwise_lambda_fn:
        # generated by the graph compiler
        alias epilogue =
        elementwise_lambda_fn.value()
        epilogue[c.dtype, width](idx, acc)
    else:
        c.store[width=width](idx, acc)
```

# Registering an Op

MOJO · REGISTER — kernels/relu.mojo

```mojo
@extensibility.register("relu")
struct Struct_relu:
    """Registers the `relu` graph op with the graph compiler."""
    @inline(.always)
    @staticmethod
    def execute[
        hidden_size: Int,          # from ops.custom parameters
        //,
        target: StaticString,      # from ops.custom device
    ](
        output: OutputTensor[dtype=.float32, rank=2, ...],
        x: InputTensor[dtype=.float32, rank=2, ...],
        context: DeviceContext,
    ) raises:
        """Applies max(x, 0) to every element of `x`."""
        comptime assert (
            Int(output.static_spec.shape_tuple[1]) == hidden_size
        ), "relu: output shape doesn't match hidden size."
        relu_kernel_api[hidden_size, target](
            output.to_tile_tensor[.int64](),
            x.to_tile_tensor[.int64](),
            context,
        )
```

PYTHON · CALL — relu.py

```python
dev = DeviceRef.from_device(device)

graph = Graph(
    "relu",
    forward=lambda x: ops.custom(
        name="relu",  # matches @register
        device=dev,     # becomes `target`
        values=[x],
        out_types=[TensorType(x.dtype, x.tensor.shape,
        dev)],
        parameters={"hidden_size": 10},
    )[0].tensor,
    input_types=[TensorType(DType.float32, [5, 10],
    dev)],
    # Mojo source dir or prebuilt .mojoc
    custom_extensions=[Path("kernels")],
)

session = InferenceSession(devices=[device])
model = session.init(session.compile(graph))
out = model.execute(x_buf)[0]
```

# Tooling

## Benchmarking & Autotuning

- Kbench includes everything from timing, throughput measurement, visualization and etc.
- Many kernels are parameterized. Autotuning finds the best configuration for a specific kernel.

### 1 Write the benchmark

```
sample.mojo
```

Parameters read at compile time: env_get_dtype, env_get_shape, env_get_int

### 2 Check it runs

```
mojo -D
shape=1024x512x256
sample.mojo
```

One config, by hand, to confirm the kernel builds and measures

### 3 Describe the sweep

```
config.yaml name,
file, params
```

Lists of dtype, shape and stages expand into a grid of configs

### 4 Benchmark

```
kbench --yaml config.yaml --output
out.csv
```

Timings table and CSV, with automatic comparison against vendor kernels

### 5 Autotune

```
kbench --yaml config.yaml --tune
```

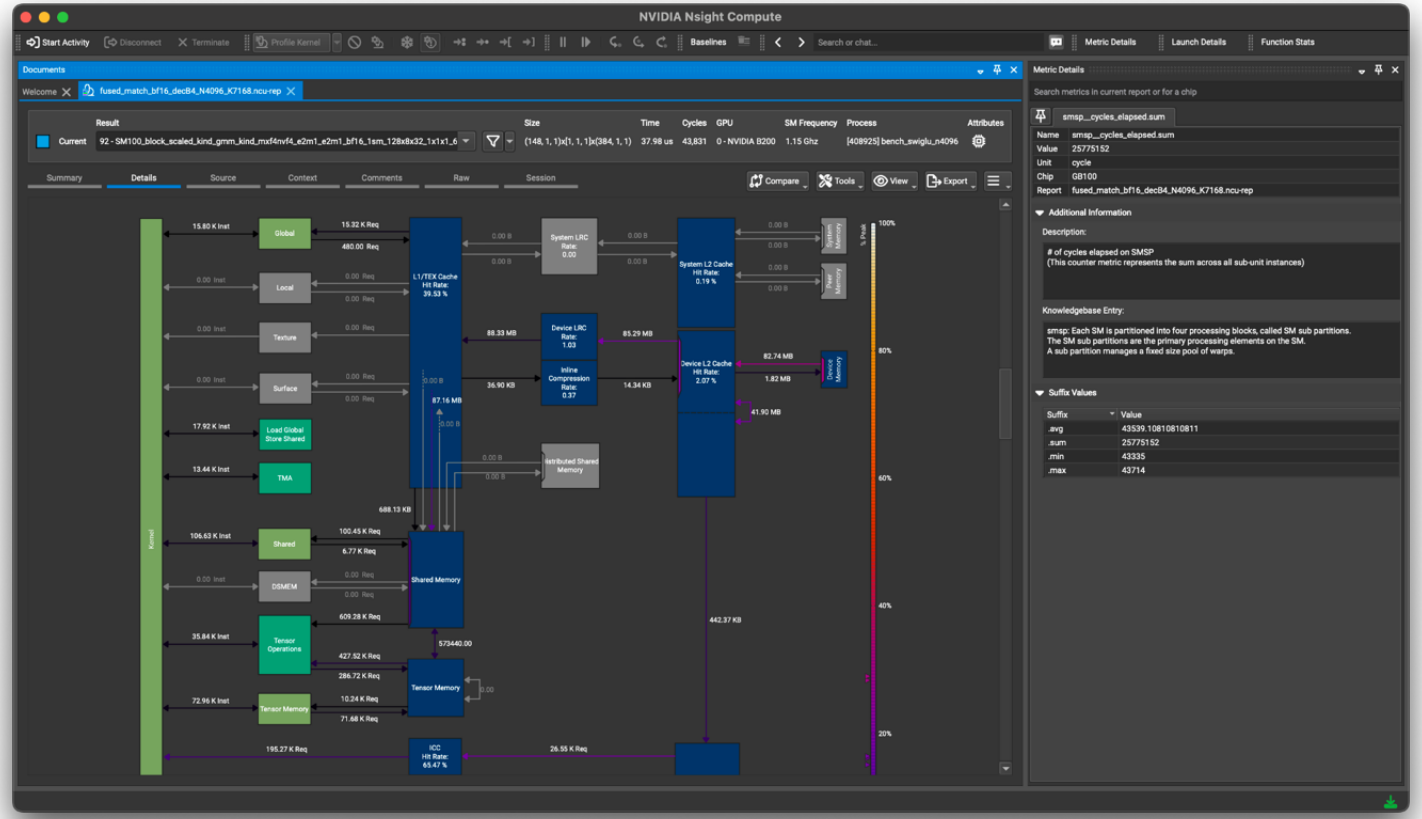
Same YAML, ranked by elapsed time, prints the best config

*One YAML file serves both.*

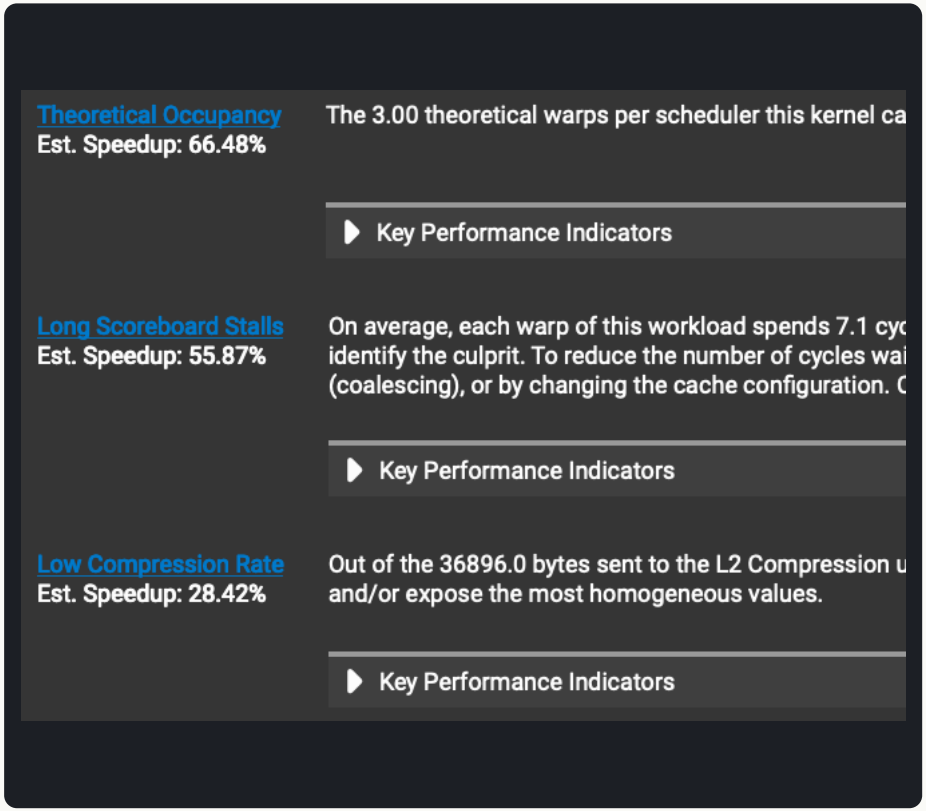
# Kernel Tools

- Performance Attribution

# Integration with existing tools



# Lack of observability to kernels




ncu's suggestions for a warp-specialized kernel, with estimated speedups of 66%, 56% and 28%

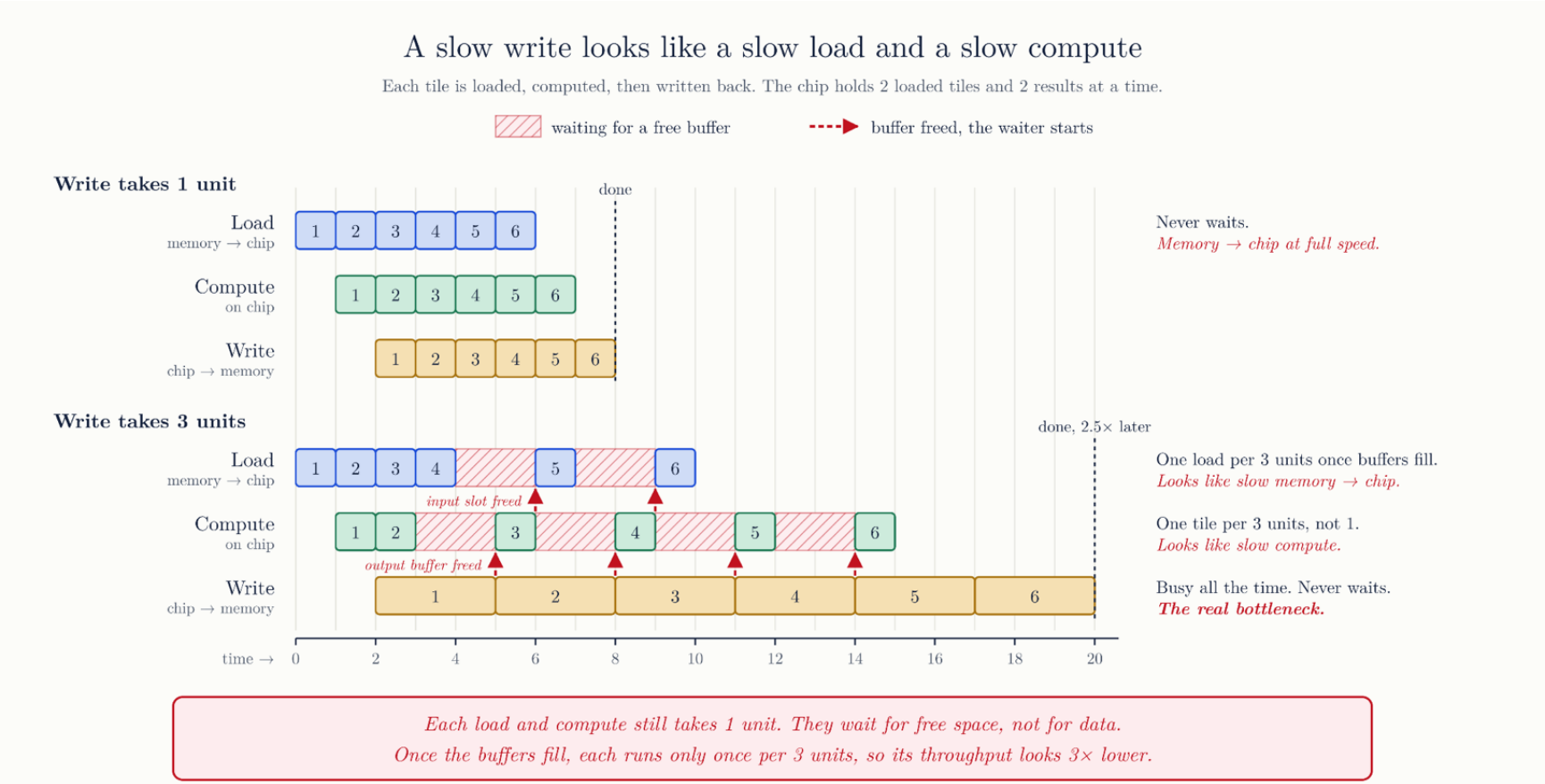
WHY

ncu fundamentally can't do performance attribution (what-if) for warp-specialized kernels without knowing how the kernel is implemented at a high level, like the data dependence graph.

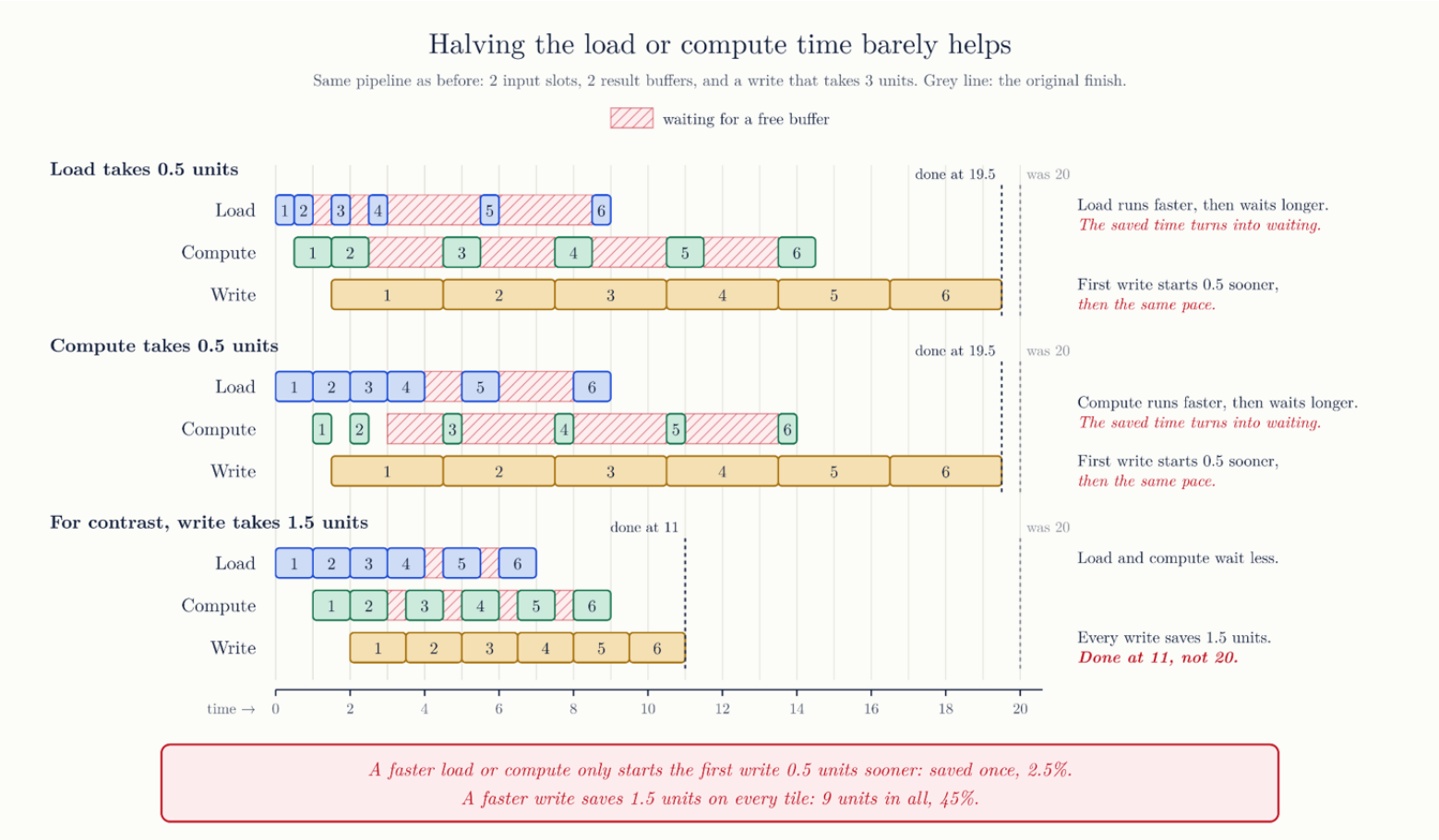
AN ANALOGY

A multi-threaded app has one thread doing the real processing and another drawing the spinner. A profiler might suggest the way to speed things up is to draw the spinner faster.

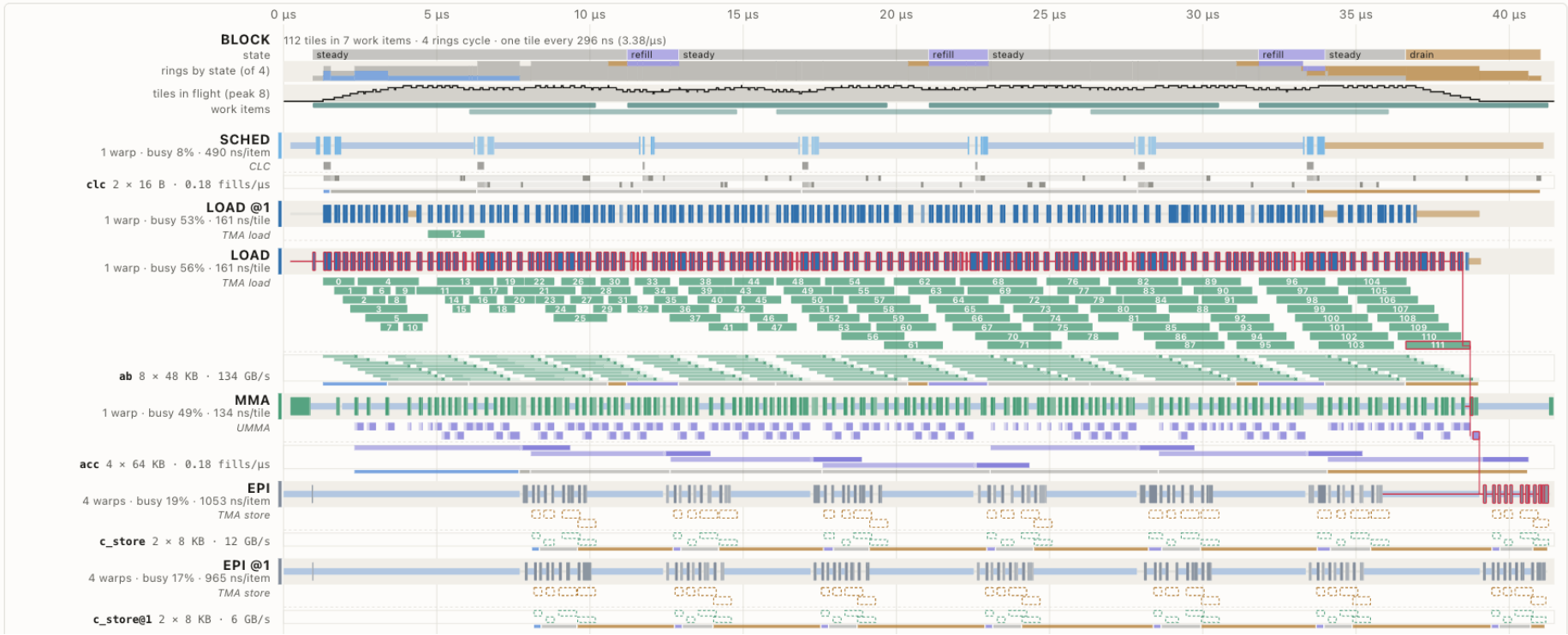
# How misattribution happens



# What-if scenarios



# Our “what-if” tool



# Our “what-if” tool

## Make something faster

Scale a class of work between 0% and 200% and the timeline above re-solves, across every one of the 1 blocks. The model reads **100%** of the measured time.[m]

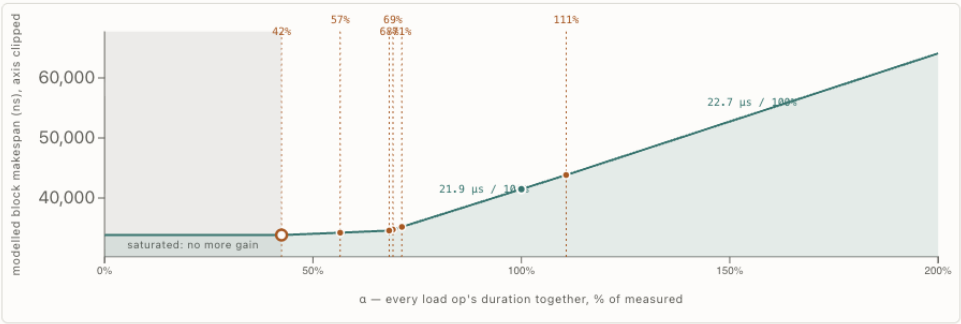
THIS BLOCK **41,472 ns** unchanged WHOLE KERNEL **42,788 ns** unchanged

BY HARDWARE UNIT | **BY WARP ROLE**

| | | |
|---|---|---|
| ▸ epi | 100 | ▾ load 100 |
| | | __FLIGHT__ab… 100 |
| | | load.ClcDeco… 100 |
| | | load.TmaLoad… 100 |
| | | load.release… 100 |
| | | load.release… 100 |
| ▸ mma | 100 | ▸ sched 100 |

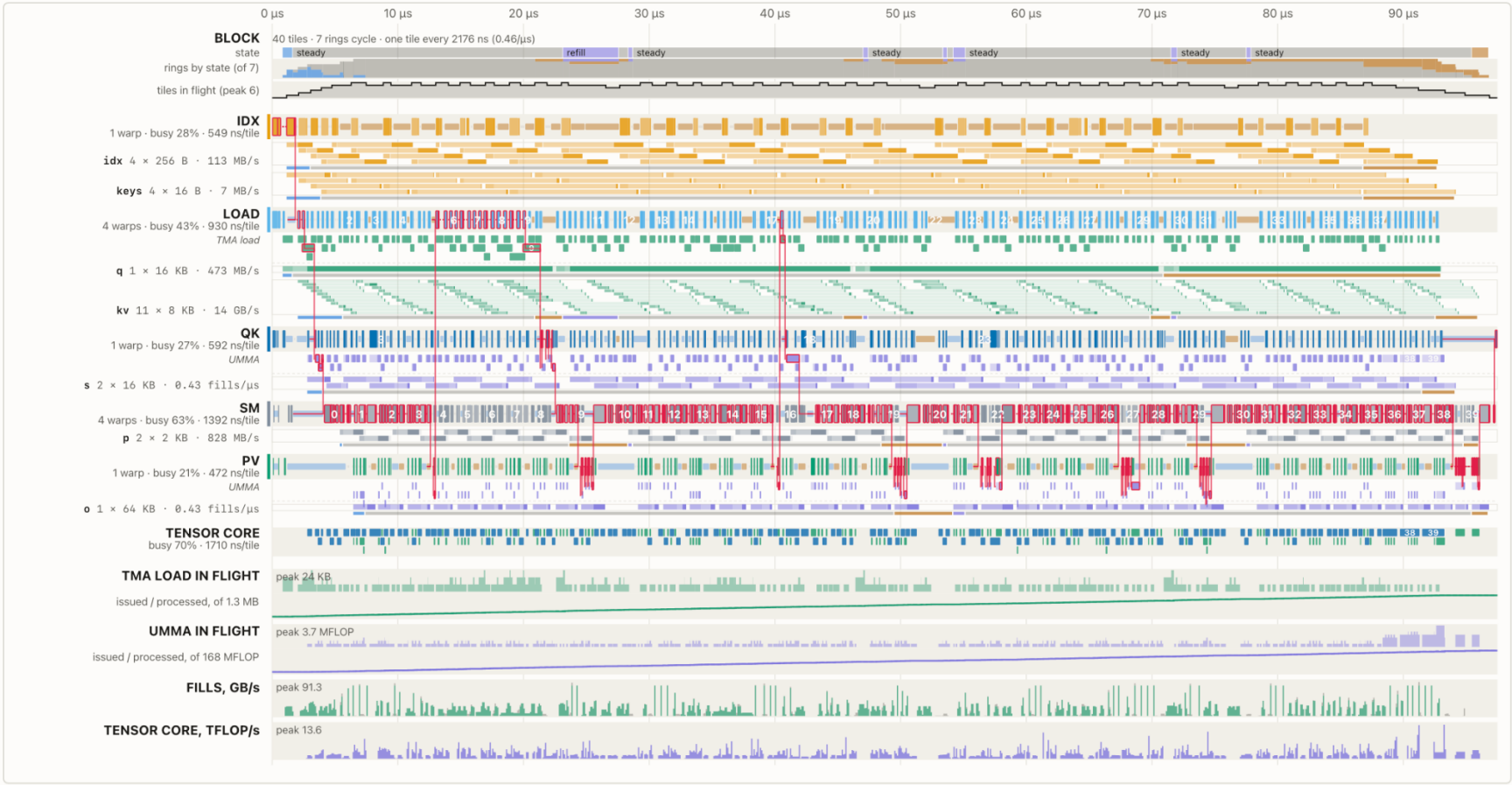
RESET EVERY SLIDER TO 100%

### Response curve — load 5 ops together

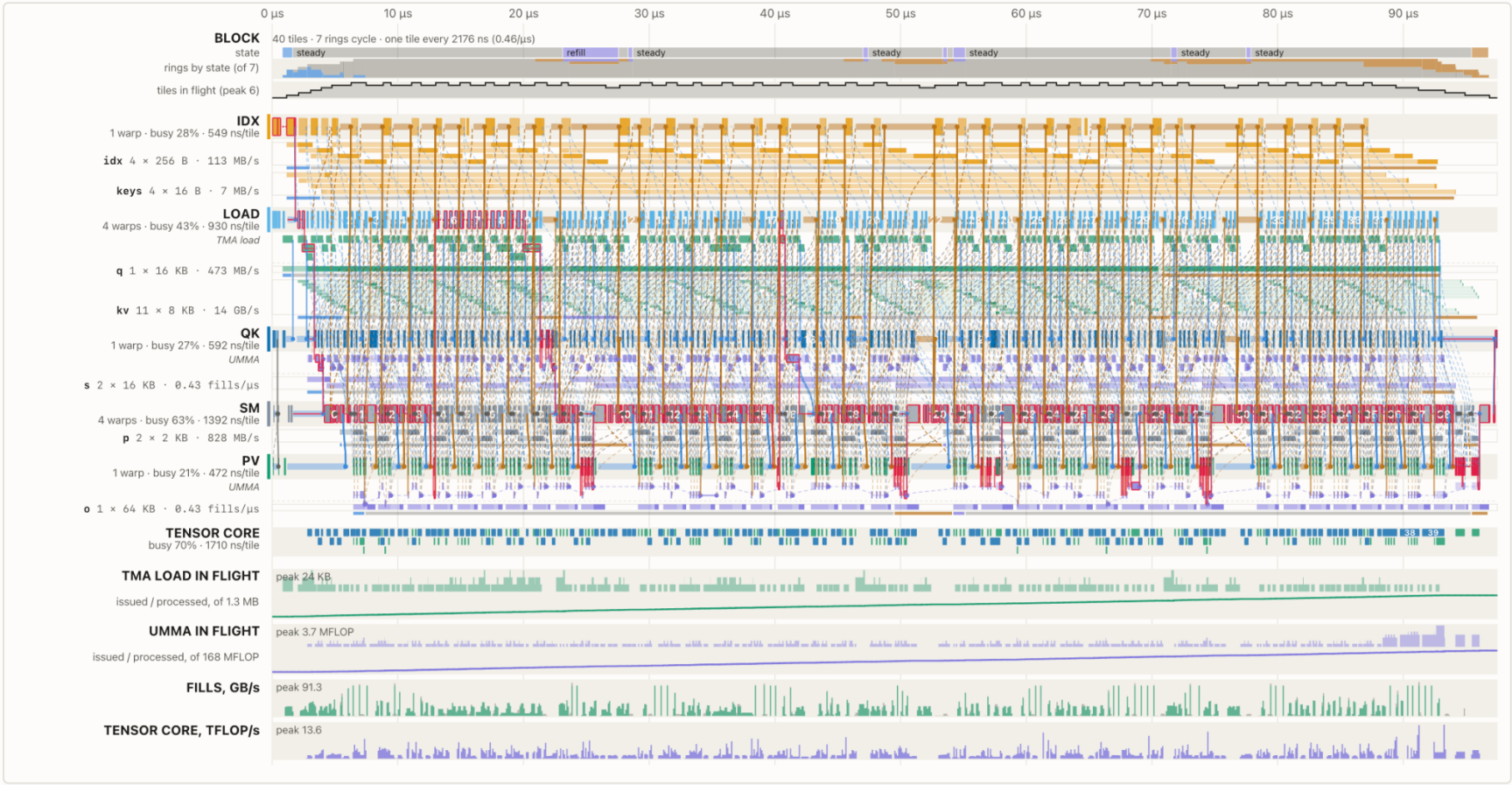



Block **0**. Making every load op free is worth **7,638 ns** of this block, from 41,472 to 33,834. Above 100% the model is on firmer ground; below it the model is an upper bound.[m] The y axis is clipped to the response and does not start at zero.

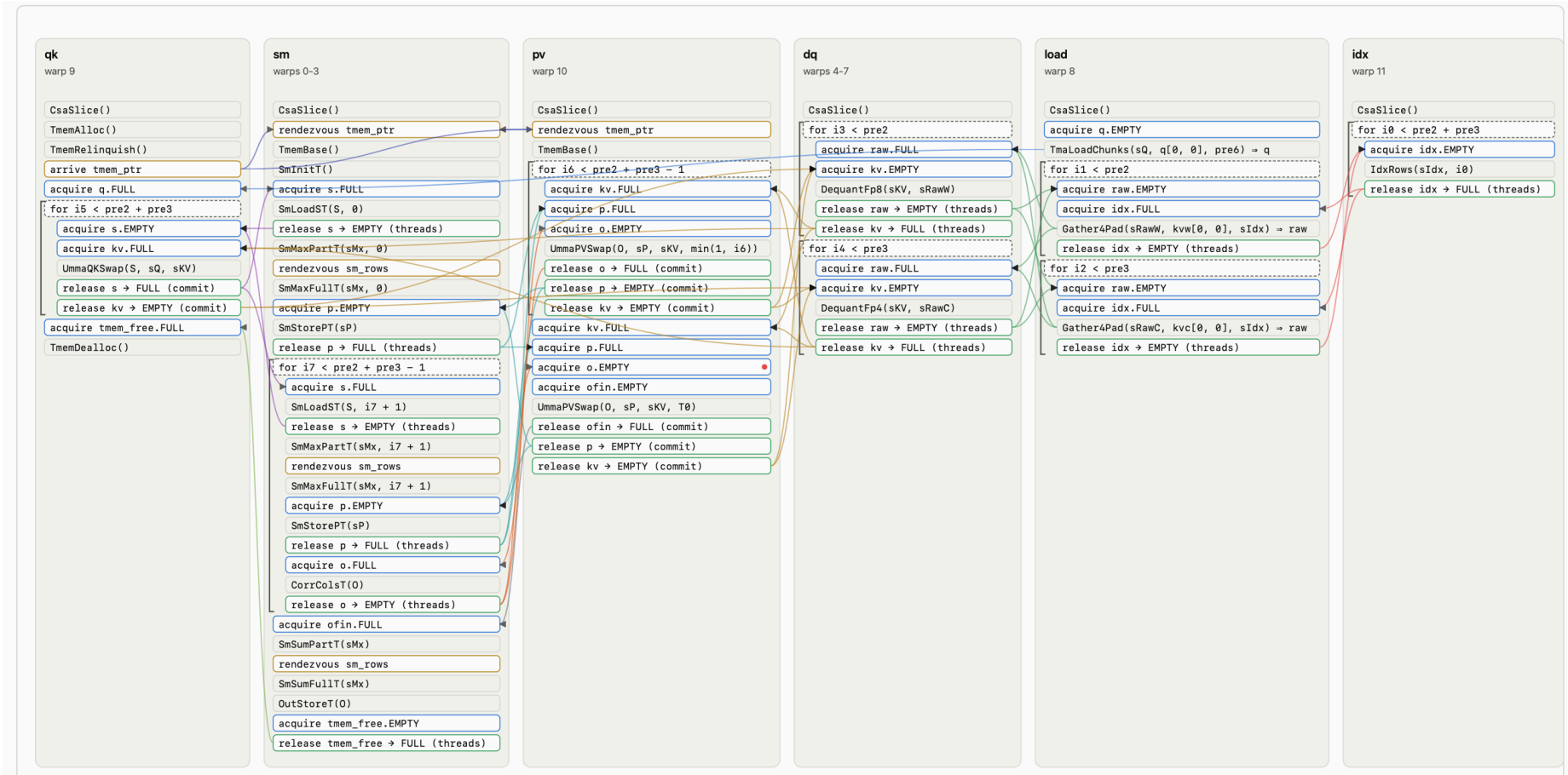
# What about a complex kernel: attention



# What about a complex kernel: attention



# What about a complex kernel: attention



# BF16 matmul vs cuBLAS

Speedup over cuBLAS. cuBLAS kernel time ÷ MAX kernel time, so values above 1.00× mean MAX is faster. Each row is a weight matrix N × K. Within ±3% counts as parity.

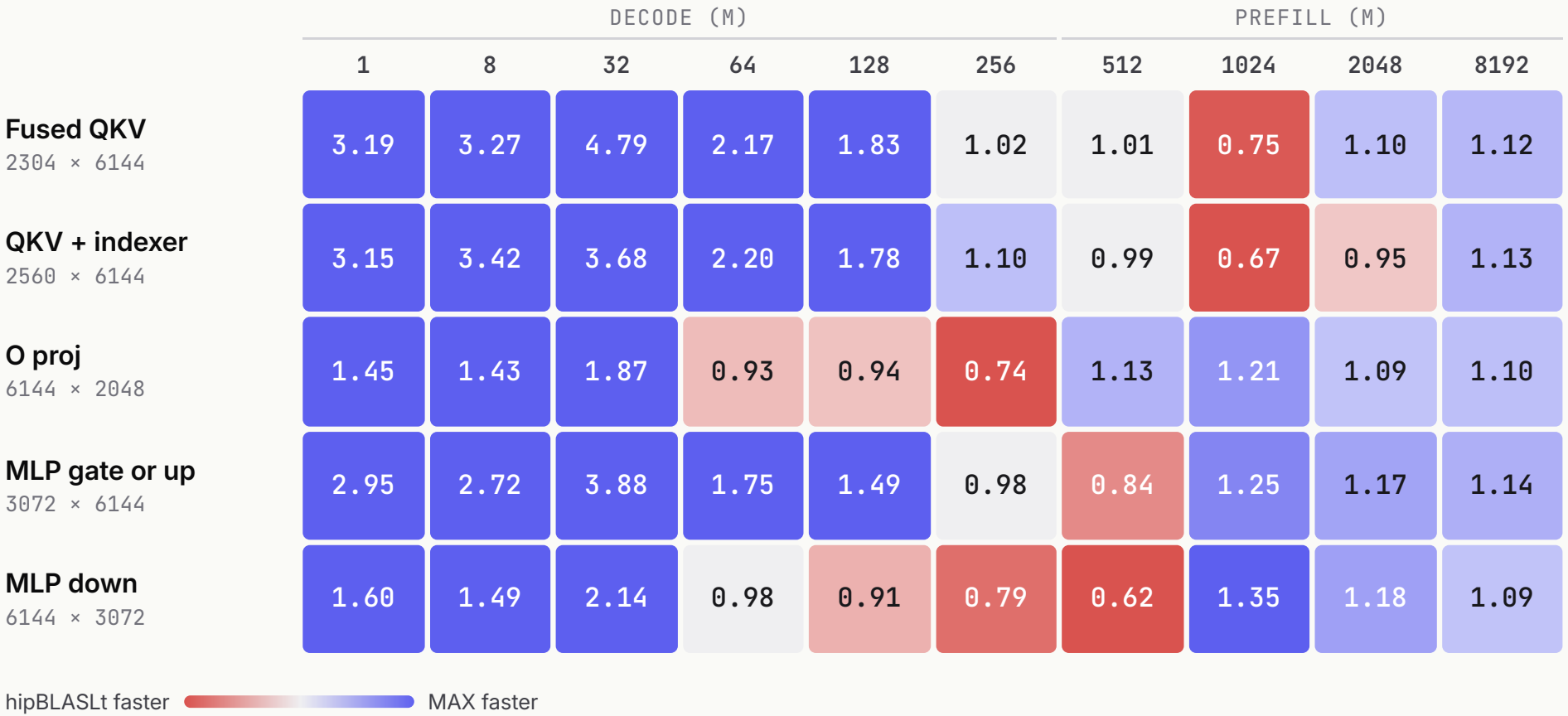
| Decode, M ≤ 128 | Prefill, M ≥ 512 | MAX vs cuBLAS |
|---|---|---|
| **1.15×** | **1.00×** | **43 / 19 / 18** |
| geomean over 40 points | geomean over 40 points | faster / parity / slower |

| | DECODE (M) | | | | | PREFILL (M) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | 1 | 8 | 32 | 64 | 128 | 512 | 1024 | 2048 | 4096 | 8192 |
| **Llama-3.1-8B QKV**<br>6144 × 4096 | 1.03 | 0.97 | 0.96 | 0.97 | 0.92 | 1.09 | 1.06 | 0.97 | 0.98 | 0.98 |
| **Llama-3.1-8B O proj**<br>4096 × 4096 | 1.19 | 1.15 | 0.95 | 0.93 | 0.89 | 1.00 | 0.96 | 0.95 | 0.97 | 0.98 |
| **Llama-3.1-8B gate + up**<br>28672 × 4096 | 1.28 | 1.17 | 1.19 | 1.12 | 1.23 | 1.00 | 1.01 | 1.05 | 0.98 | 0.95 |
| **Llama-3.1-8B down**<br>4096 × 14336 | 1.46 | 1.26 | 1.22 | 1.27 | 1.29 | 1.05 | 0.96 | 0.94 | 0.95 | 1.03 |
| **Llama-3.3-70B QKV**<br>10240 × 8192 | 1.35 | 1.20 | 1.25 | 1.09 | 1.18 | 0.96 | 1.11 | 1.04 | 1.00 | 1.06 |
| **Llama-3.3-70B O proj**<br>8192 × 8192 | 1.20 | 1.19 | 1.17 | 1.13 | 1.16 | 0.96 | 1.00 | 1.04 | 0.99 | 1.00 |
| **Llama-3.3-70B gate + up**<br>57344 × 8192 | 1.18 | 1.15 | 1.08 | 1.06 | 1.21 | 1.04 | 1.05 | 0.98 | 0.98 | 0.97 |
| **Llama-3.3-70B down**<br>8192 × 28672 | 1.09 | 1.20 | 1.30 | 1.33 | 1.19 | 0.99 | 0.99 | 1.01 | 0.97 | 0.94 |

cuBLAS faster — MAX faster 0.74× · 1.00× · 1.35×

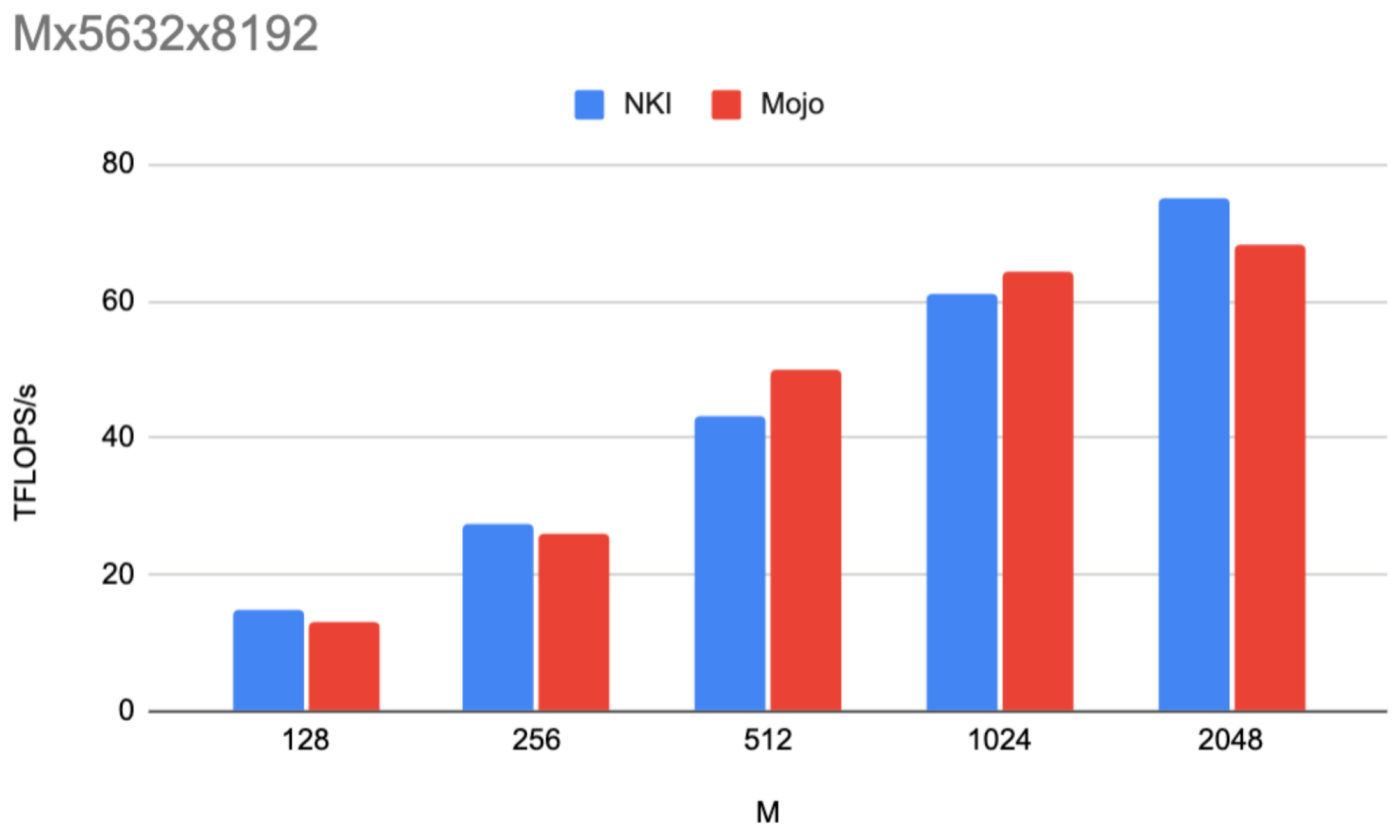
# MXFP8 matmul vs hipBLASLt (MiniMax-M3 Shapes)

MXFP8 vs hipBLASLt

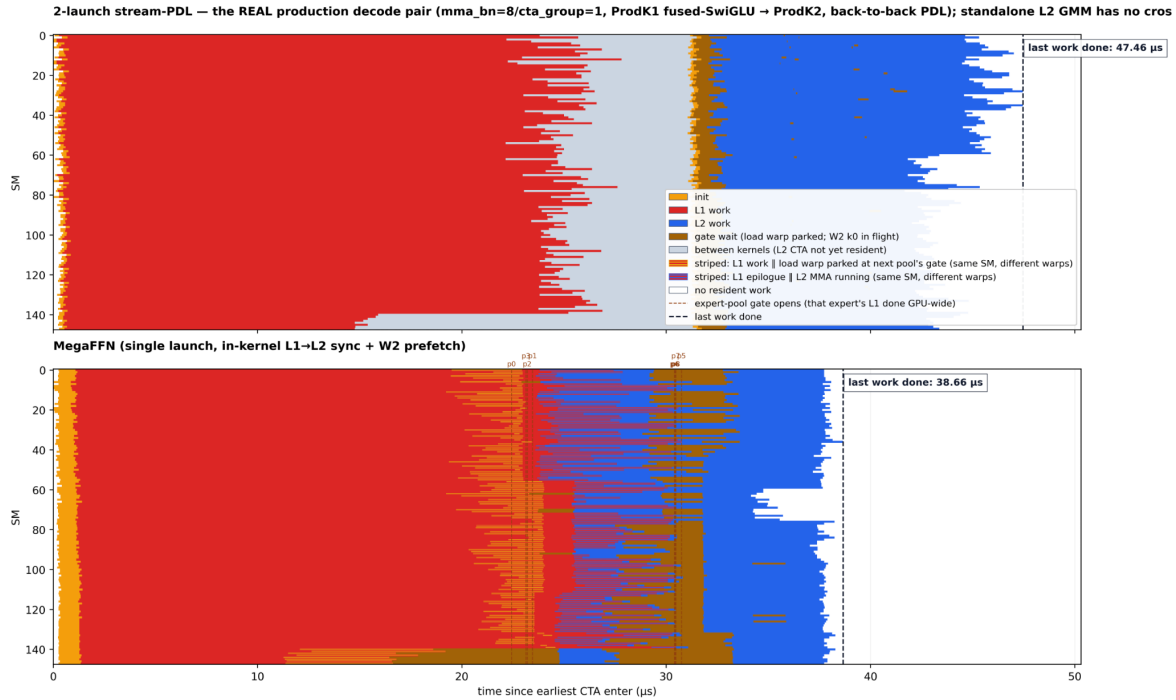


# Trainium BF16 matmul vs NKI

Gemma4 shape



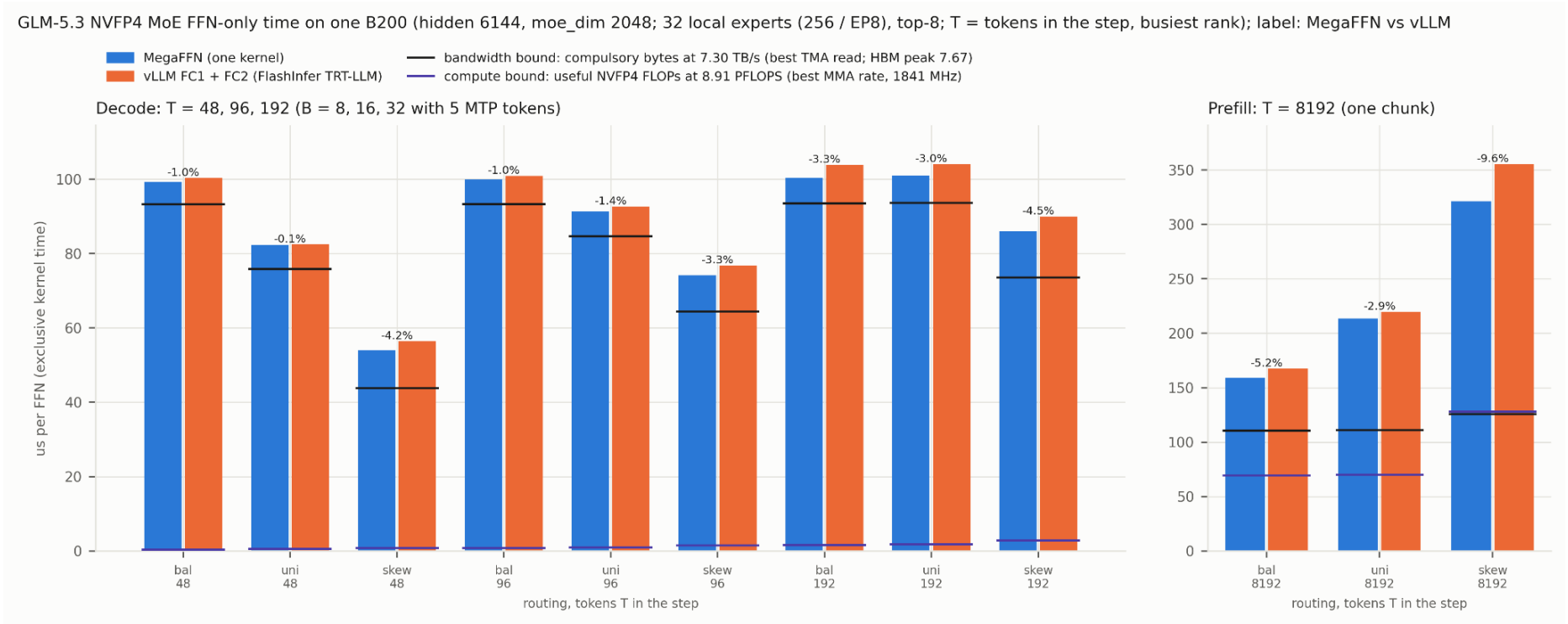
# MegaKernel



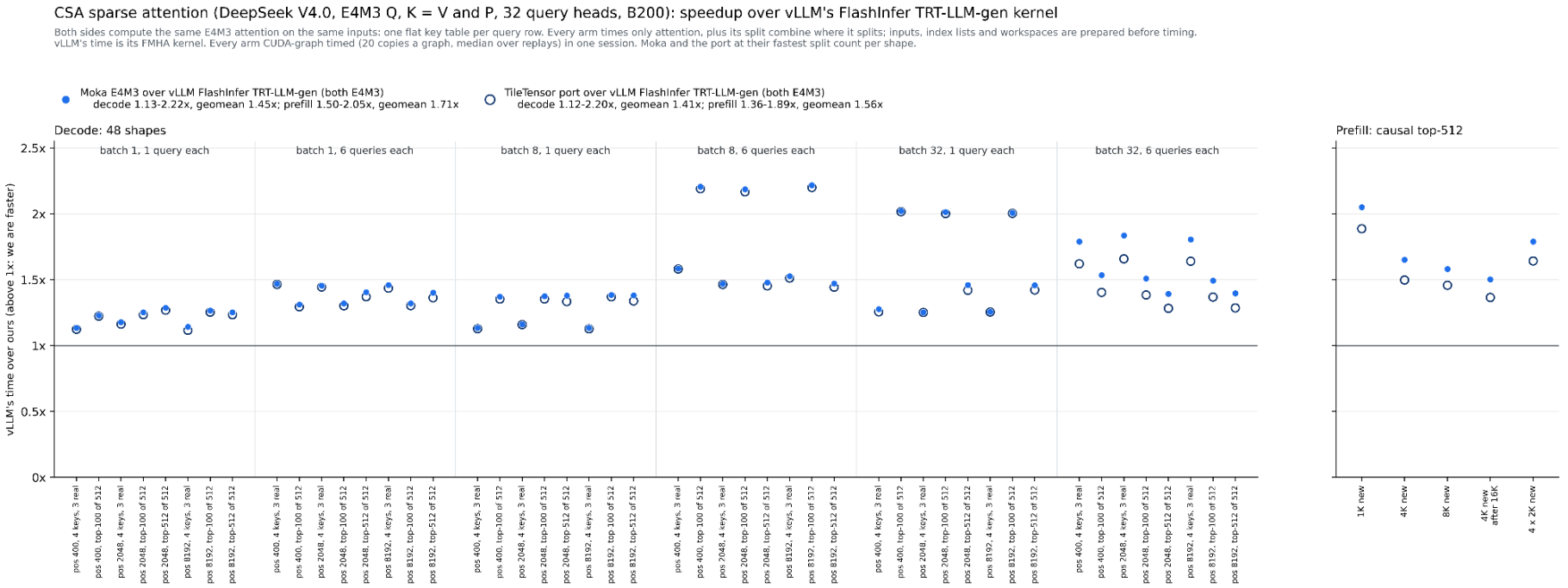

## MegaFFN =

- fused grouped matmul for up | gate
- SwiGLU
- quantization to MXFP8/NVFP4
- grouped matmul for down proj

# Grouped matmul vs vLLM kernels

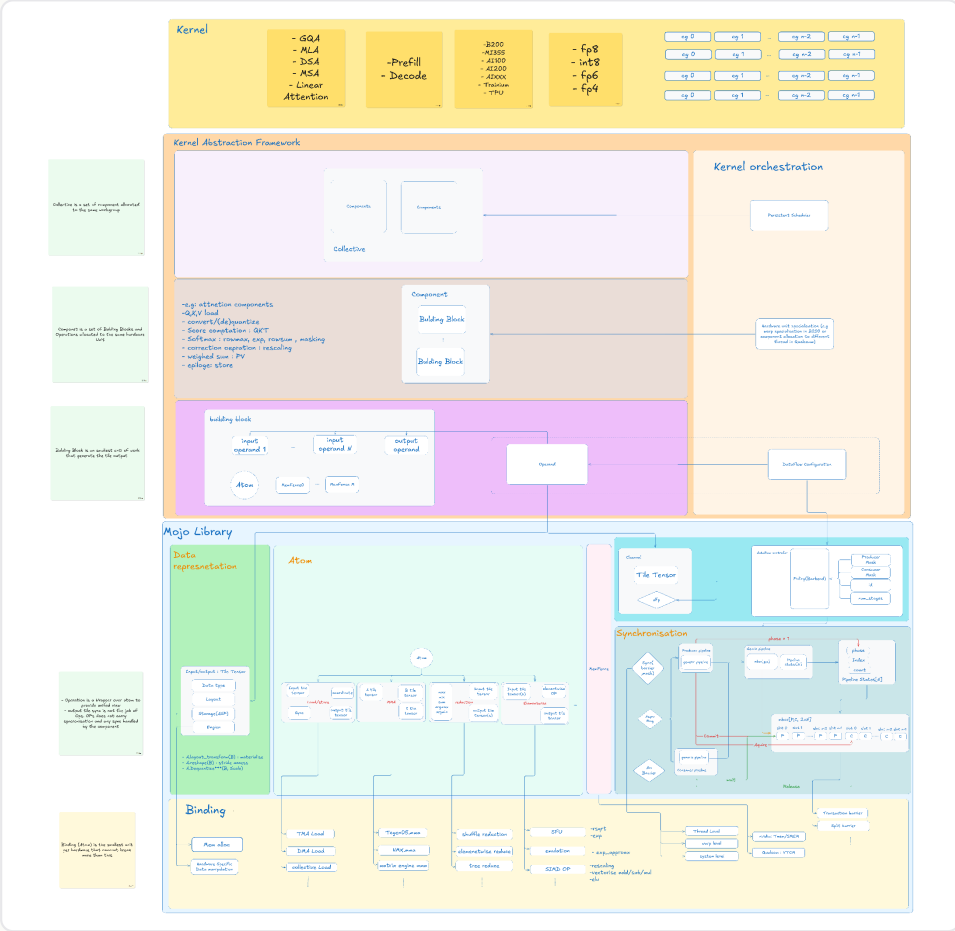


# CSA sparse attention vs vLLM kernels



# Kernel abstraction: separation of concerns

- **Orchestration**
  - Scheduling
  - Resource management
- **Components**
  - Atoms: TMA, DMA, MMA etc
  - Copy, mma, rowmax, softmax



# Accelerate kernel authoring via agentic tooling

Agent tooling for kernel work in MAX

SKILL · AGENT · CLI TOOL · WORKFLOW

## Know the platform CONTEXT

- **mojo-syntax** current Mojo syntax and conventions
- **mojo-gpu-fundamentals** how Mojo GPU code is written
- **mojo-kb** Kernels KB: 251 checked entries on TileTensor, MMA/MFMA, pipelining, swizzles
- **amd-gpus** CDNA 4 / MI355X constants: LDS, MFMA shapes, peaks

## Pick the target SHAPES · COMPETITION

- **model-kernel-map** which kernels a model runs, and which models call a kernel
- **extract-kernel-shapes** real per-rank shapes (TP/EP/DP) for 17 models; benchmarks against vLLM/SGLang kernels
- **competitor-researcher** reads FlashInfer, CUTLASS, DeepGEMM, AITER… with file:line citations

## Write and verify CORRECTNESS

- **mojo-kernel-engineer** writes or ports kernels; checks against a reference before any perf claim
- **add-kernel-fuzz-target** edge-shape, NaN and determinism fuzzing (37 targets)
- **debug-gpu-issues** hangs, illegal memory access, NaN/Inf, wrong logits
- **kernel-PR-reviewer** OOB, barriers, spills, bank conflicts, test coverage

## Benchmark SPEED

- **kbench** kernel benchmark and autotune sweeps
- **kepler** microbenchmarks with time / ncu / nsys, local or remote B200
- **bench-kernel-pr** runs kernel benchmarks for a PR on H100, B200, MI355
- **measurement-integrity** is the path live and the machine quiet before trusting a number

## Profile WHY IT'S SLOW

- **ncu-analyze** Nsight Compute: occupancy, throughput, stalls
- **kernel-profile-evidence** fixed-schema JSON: bottleneck class, distance to SOL
- **nsys-profile · rocm-profile** timelines on NVIDIA and AMD
- **mojo-compile-timing** where a Mojo compile spends its time

## Close the loop AUTONOMOUS

- **model-opt-loop** benchmark-gated optimization from serving down to kernels; commits each accepted step
- **kernel-optimizer** profiles, then makes one evidence-backed change per round
- **compiler-engineer** when the slowdown is codegen or fusion, not the kernel
- **colleague** second model from another vendor that checks results for cheating

# Where are we heading

- Kernel abstraction
- Megakernels and mega fusions
- Expanding Qualcomm HW support: AI200, AI250, and beyond

# Where to learn more?

### Mojo GPU puzzles

https://puzzles.modular.com

### MAX and Mojo docs

https://max.modular.com

https://mojolang.org

### go/ModularTechStack

Previous talks in this series and more

Modular